# Tackling Algorithmic Bias

*Ross Jenkins*

4th Year Project Report
Computer Science
School of Informatics
University of Edinburgh

2019

# Abstract

As algorithms continue to penetrate our daily lives, it's becoming imperative that as a community we possess the awareness of the problem and a toolset to solve it. In this report, I provide a taxonomy of Algorithmic Bias, including a consise definition and a framework of its common causes. Following which, the reader will find an analysis of various means proposed in literature to quantify and mitigate algorithmic bias along each stage of the classification pipeline. The latter portion of this report is composed of a Case Study, whereby I apply the quantification metrics and bias mitigation methods discussed above in order to provide some perspective to the reader of each method's effectiveness.

# Acknowledgements

Firstly, I'd like to dedicate this report to the memories of my Grandfather, William James Jenkins, and my two Great Grandmothers, Helene Orbinson & Annie Jenkins. Each of whom inspired me as I grew up with their wit, intelligence, and strength. They are dearly missed.

I'd also like to give my thanks and appreciation to my Project Supervisor, Dr. Michael Herrmann. For his guidance and understanding over the course of this year, and for the great conversations we've had in your office.

Lastly, I'd like to thank my girlfriend, Amber Louise Clarke and my parents, Jill Jenkins & Simon William Jenkins. For their endless love and support throughout my degree, and especially during what has been a very difficult period.

# Table of Contents


**1 Introduction** **1**
- 1.1 Motivation . . . 1
- 1.2 Overall Aims & Objectives . . . 1
- 1.3 Structure of Report . . . 2

**I A Conceptual Report on Algorithmic Bias** **3**

**2 What is Algorithmic Bias & How is it Caused?** **5**
- 2.1 What is Algorithmic Bias? . . . 5
- 2.2 How is Algorithmic Bias Caused? . . . 6
  - 2.2.1 Propagation/Amplification of Historical Bias . . . 6
  - 2.2.2 Bias Recursion . . . 7
  - 2.2.3 Sample Bias . . . 8
  - 2.2.4 Transfer Context Bias . . . 8
  - 2.2.5 Incomplete Data Bias . . . 8
  - 2.2.6 Summary . . . 9

**3 Quantifying Discrimination and Mitigating Algorithmic Bias** **11**
- 3.1 Quantifying Discrimination to Diagnose Potential Bias . . . 11
  - 3.1.1 Disparate Impact . . . 12
  - 3.1.2 Statistical Parity Difference . . . 12
  - 3.1.3 Generalised Entropy Indices . . . 13
- 3.2 Feature Suppression . . . 14
- 3.3 Pre-processing Methods to Mitigate Bias . . . 15
  - 3.3.1 Disparate Impact Remover . . . 16
  - 3.3.2 Reweighing . . . 16
- 3.4 In-processing Methods to Mitigate Bias . . . 17
  - 3.4.1 Adversarial Debiasing . . . 17
- 3.5 Post-processing Methods to Mitigate Bias . . . 17
  - 3.5.1 Calibrated Equalised Odds Postprocessing . . . 18
  - 3.5.2 Reject Option Classification . . . 18

**II A Case Study: How to Quantify and Mitigate Algorithmic**

**Bias** **19**

**4 Methods** **21**
4.1 The Dataset Used . . . . . 21
4.2 Pre-processing Steps Applied to Data . . . . . 22
4.2.1 Feature Identification . . . . . 22
4.2.2 Handling Null Values . . . . . 23
4.2.3 Reducing Data with Low Value Counts . . . . . 24
4.2.4 One-Hot Encoding . . . . . 25
4.2.5 Handing Imbalanced Classes using SMOTE . . . . . 25
4.3 Data Exploration . . . . . 26
4.4 Scikit-learn . . . . . 29
4.4.1 Models Used . . . . . 29
4.5 AI Fairness 360 . . . . . 31

**5 Outcomes** **35**
5.1 Baseline Performance & Fairness of Models . . . . . 36
5.2 Feature Suppression . . . . . 37
5.3 Application of Pre-processing Bias Mitigation . . . . . 38
5.3.1 Reweighing . . . . . 38
5.3.2 Disparate Impact Remover . . . . . 39
5.4 Application of In-processing Bias Mitigation . . . . . 41
5.4.1 Adversarial Debiasing . . . . . 41
5.5 Application of Post-processing Bias Mitigation . . . . . 42
5.5.1 Calibrated Equalised Odds Post-processing . . . . . 42
5.5.2 Reject Option Post-processing . . . . . 42

**6 Conclusion** **45**

**7 Future Work** **47**

**Bibliography** **49**

# Chapter 1

# Introduction

## 1.1 Motivation

Every day, automated decision making processes and the algorithms they employ make millions of decisions. From which advertisements are shown on the websites you visit [26] and if you'll get the jobs you apply for [10], to how resources are allocated across hospital networks [8] and if someone accused of a crime is a risk to public safety [2].

We often expect and rely on such processes as being impartial, objective arbiters that make decisions without being influenced by subjective factors in the ways that humans are. However, in reality, algorithms often end up propagating and sometimes amplifying human biases when applied without auditing from the developer.

With the recent enaction of the GDPR across the European Union, and conversations between industry and legislators happening across the world, the problem of algorithmic bias is becoming more recognised [8, 29]. Developers need to ensure that they react by knowing the causes, effects and how to quantify and mitigate bias in their systems as we continue to rely both on increased volume and complexity of the decisions they make.

Hence, the development of fair automated decision making processes requires recognition of the problem, commitment to change the way we develop AI, and developing the tools required to audit our models as they evolve.

## 1.2 Overall Aims & Objectives

In this report I will provide to the reader a concise overview of what algorithmic bias is, and a framework for the common causes of algorithmic bias. This will be followed by an analysis of how we can quantify discrimination in the context of automated decision making, and how to mitigate algorithmic bias along each stage of the decision-making pipeline.

Later in the report I will provide a case study on how one can use the methods described to quantify and mitigate potential bias on an existing dataset. This will give the reader some perspective of how one can audit their automated decision making processes for such potential biases.

## 1.3 Structure of Report

As alluded to above, the following report will be split into two parts: a Conceptual Analysis on the Subject of Algorithmic Bias, followed by a Case Study - wherein I apply the methods of bias quantification and mitigation discussed in the earlier part.

Included in my analysis of Algorithmic Bias will be a discussion of each of the following topics:

- What is algorithmic bias?
- How is algorithmic bias caused?
- Quantifying Discrimination, and Mitigating Algorithmic Bias

The second part of this report will consist of a case study on an existing dataset, wherein I apply the methods of bias quantification and mitigation discussed in Part 1.

After introducing the dataset used, I will describe the pre-processing steps I applied and introduce a range of standard models used to classify the data. Following this I will provide an interpretation of the non ”fairness-aware” classifications, looking at the performance of each model in terms of classification accuracy, precision, and the Area Under the ROC curve for each model. At this point, I’ll also quantify any perceived levels of bias picked up by each model in terms of the metrics discussed in Part 1.

Next, I will apply the methods of bias mitigation discussed in Part 1 along each stage of the classification pipeline. Said pipeline has three stages:

- Pre-processing
- In-processing
- Post-processing

After which, I will provide an analysis of the results in terms of the trade-off between performance and fairness. This will offer the reader some perspective of a range of methods for bias mitigation that exist, and how well they can each be expected to perform in practice.

# Part I

# A Conceptual Report on Algorithmic Bias

# Chapter 2

# What is Algorithmic Bias & How is it Caused?

## 2.1 What is Algorithmic Bias?

We can define an algorithm as possessing a bias where it can be shown that it is systematically producing unfair outcomes against groups of people based upon their common characteristics, rather than on their individual merits. Despite our belief in the contrary, algorithms can exhibit biases that reflect human biases against typically marginalised groups such as women, people of black or minority ethnic (BAME) descent, and members of the LGBT community [2, 10, 26].

Hence, in the context of this report, an algorithm is "*biased*" if it manifests a *systematic and unfair, discriminatory outcome* on marginalised groups such as those outlined above.

The characteristics that define such groups to be protected from discrimination are often well-defined in anti-discrimination legislature. For instance in the UK's Equality Act (2010) [1] there is a set of nine "Protected Characteristics" (otherwise known as Protected Attributes) for which discrimination against is unlawful, these are as follows:

- Age
- Disability
- Gender Reassignment
- Marriage or Civil Partnership
- Pregnancy and Maternity
- Race
- Religion or Belief
- Sex
- Sexual Orientation

It is therefore unlawful for one to employ an algorithm that can be shown to exhibit a bias in outcome based upon any of the above protected characteristics. It's also useful to note here that whilst human decision makers can consciously or unconsciously make biased decisions contextually, computers lack any subjective knowledge. It is the by virtue that biased models make discriminatory decisions *continuously and systematically* that is most damaging.

## 2.2 How is Algorithmic Bias Caused?

It is easy to mistake the algorithms and the wider system used to make decisions computationally as being controlled by the designer, however, in practice it is often hard to tell exactly how the algorithms make their decisions. This is especially true in cases when the system is increasingly complex, for example when outputs of multiple algorithms are fed into each other or when data is high-dimensional. In such cases it can become impossible to diagnose the source of bias.

In reality, algorithms build a model used to make decisions based upon the patterns it learns from the *data itself*, so the designer often doesn't have much control on the outcome. Hence, biased algorithms are typically the outcome of the data used to build the model.

So how are the models obtained? Given that we have a dataset containing a range of attributes relating to a person, both sensitive and otherwise, and we have some kind of target attribute that we would like to classify each person under. We then *train* an instance of one (or many) of the classification algorithms developed using *training data*. This is a dataset such that that we possess the observed outcomes, typically a subset of the wider data available to the designer. To obtain a model is to apply a given classification algorithm to a given training dataset. After a model is learned, then it can be used to classify instances such that the outcome is unknown.

There are many ways posited in which we can categorise sources of algorithmic bias, which I will attempt to provide a framework for below:

### 2.2.1 Propagation/Amplification of Historical Bias

Given the understanding that the algorithms we use to classify data is built upon the training set that we give them, it is easy to understand that if we use data encapsulating an historical human bias, that the resulting model will propagate and in some cases amplify such biases.

As outlined above, the training data which we use to build models is a collection of observed instances *and* their associated outcomes. This introduces two possibilities for the propagation of historical bias into our models:

#### 2.2.1.1 Representational Bias

If an historically marginalised group is not represented in the training data used to build a model, then the model will learn to discriminate against such a group. For example, if a model is trained to classify if a person has the characteristics of a CEO or not and is trained on a historical database of CEOs (a similar situation to the bias shown in [17]), the likelihood of a woman or member of another historically marginalised group being classified as a CEO becomes very slim.

This is a result of a historical, human bias affecting marginalised groups being manifested in the decision making of an algorithm. In this case, without treatment, any model trained on such historically biased data will discriminate against women by not classifying them based upon their individual characteristics but by their gender. In training, the model will have learnt a strong link between being male and being a CEO.

#### 2.2.1.2 Outcome Bias

However, there are also cases where protected groups *are* represented in the training data but it's the *outcomes* that are biased against them. A similar outcome results where the model learns a latent discriminatory link between a protected characteristic and a negative outcome. For example, if a model is trained to classify whether a person will be admitted to University or not is trained on historical outcomes of applications (as described in possibly the first recorded case of algorithmic bias in [24]). The model can become discriminatory against historically marginalised groups such as women or ethnic minority groups, as it learns a link between their characteristics and a negative outcome.

Given that the data represents a history of not giving positive outcomes to members of such groups - which is another example of a common, historical, human bias - the model will learn a discriminatory link between their protected characteristics and a negative outcome.

To be clear, the distinction between the two biases outlined above come from the following standpoints:

- Representational Bias is characterised as an *emergent* bias, resulting from a lack of historical examples of *positive* outcomes for a protected group.
- Whereas Outcome Bias is characterised as a *latent* bias, resulting from a distinct amount of historical examples of *negative* outcomes for a protected group.

### 2.2.2 Bias Recursion

In cases where models are built upon training data that has been subject to historical/societal bias, a "*feedback loop*" can be formed as a model becomes more confident in its biased models through their confirmation. As described by Calders & Zliobaite in [6].

This is a situation where a model's bias is amplified by a *self-fulfilling prophecy*, whereby biased data based upon discriminatory assumptions result in biased outcomes that stand to reinforce the original historic/societal bias.

For instance, given a system is designed to offer predictive policing based upon crime rates and that police presence is typically heightened in majority black communities [23]. It is obvious that crime rates will be higher in such areas by virtue of the fact that more police officers are in the area, not necessarily because less crime occurs in other areas. However, the model learns based off of the data it possesses. A bias against majority black communities is nonetheless reinforced, as the system continues to allocate more resources and becomes more confident in its model.

### 2.2.3 Sample Bias

This describes a situation in which the training data compiled and used to build a model is not representative of the wider context of the algorithm's application, resulting in bias through omission or under-representation of groups that the system should be accounting for. It's often difficult for system designers to ensure that their training sets are large and well-balanced enough to make meaningful predictions where the system domain is wide, but it's key to ensuring that there are no opportunities for bias to creep in.

Across the tech industry, real life examples exist where facial recognition software has decreased accuracy depending on race and gender [27, 28], often in circumstances where the models that they train aren't being fed enough instances of minority ethnic groups or females. Simply put, the use of unbalanced training sets results in bias.

### 2.2.4 Transfer Context Bias

As described by Danks & London in [9], *transfer context bias* can arise from the inappropriate use or deployment of a system outside of the context in which it was trained, based on the assumption that it can still perform as necessary. For instance, if we train a model to predict if someone will pay back a loan using data from an urban area and attempt to deploy the model in a rural area. As both the characteristics of the people, and the economy of the region differ dramatically, the model may exhibit a bias against those in the rural area.

### 2.2.5 Incomplete Data Bias

As described by Calders & Zliobaite in [6], absence of important characteristics, can result in bias. Often due to privacy/legal concerns, or because the data itself is difficult to collect, attributes that are important to predicting outcome are missing. Where such ideal attributes are missing, the model is built upon maximisation of accuracy based

upon the attributes it does possess, resulting in predictions based off of *proxy* attributes for the missing data that the model learns are linked to positive/negative outcomes.

For instance, to train a model to predict how high a person's car insurance premium is, the ideal dimension to have is whether or not a person is an aggressive driver or not. However, this data is difficult to obtain. The model may then learn that males often have higher premiums, which becomes a proxy for the more ideal attribute of the person's driving style. Resulting in males being more likely to be classified as high risk regardless of their driving style.

### 2.2.6 Summary

Overall, the situation can be summarised by the phrase "*garbage in, garbage out*". When developing algorithmic decision making processes, it is imperative that the training data used is representative and minimises any possible latent or emergent bias that can be propagated to and exacerbated by the models produced.

Additionally, it can be said that given that the incentive of the aforementioned models is to maximise classification accuracy with no respect to potential bias toward specific dimensions in the data/fairness in general, bias becomes a by-product of using training data that exhibits any forms of sample bias or encapsulates any form of historical bias.

Given the above situation, the burgeoning field of "fairness-aware AI" has sprung, and a breadth of research is being carried out into algorithmic fairness methods for developers to utilise in the process of diagnosing and mitigating algorithmic bias. This field provides much of the context for discussion in Chapter 3.

# Chapter 3

# Quantifying Discrimination and Mitigating Algorithmic Bias

There are two steps to take when auditing algorithmic decision making process, firstly we quantify any bias against groups within the test data based on their sensitive attributes (i.e Race, Gender), and then we attempt to reduce this quantity through the application of bias mitigation techniques along the classification pipeline whilst retaining as much performance as possible. There are three points along the pipeline that one can apply bias mitigation techniques:

- **Pre-processing**: adjusting the training data *before* training a model.
- **In-processing**: producing a "fairness aware" model that learns fair parameters *whilst being trained*.
- **Post-processing**: *adjusting predictions made* by the model after the fact with respect to fairness.

## 3.1 Quantifying Discrimination to Diagnose Potential Bias

There are a range of metrics that one can use to quantify potential bias whilst auditing the algorithms used to make decisions. Here I will discuss two quantities related to the proportion of favourable outcomes measured for the privileged and unprivileged groups, and a further quantity related to measuring the generalised fairness both between these groups and within the groups themselves.

Hence, two important concepts to consider in the pursuit of algorithmic fairness are *group fairness* and *individual fairness*. Group fairness defines the requirement that privileged and unprivileged groups should be treated similarly, or groups within a population should have proportionately similar outcomes compared to the whole. Individual fairness defines the requirement that all individuals should be treated according to their merit and not on their characteristics. Taking this into account, bias in an automated decision making process can be shown through metrics related to group fairness,

individual fairness, or both. We seek to develop and maintain processes that provide us with both fairness with respect to unprivileged groups of people *and* individuals across the entire population.

### 3.1.1 Disparate Impact

The Disparate Impact (DI) score (related to the legal concept described in [3]) is a group fairness metric measured by the *ratio* of the proportion of favourable outcomes for the unprivileged groups to the proportion of favourable outcomes for the privileged group. Hence, a score of less than 1 implies a benefit in favour of the privileged group and a score of greater than 1 implies the opposite. An "ideal" score would be 1, as this would imply a situation where there is no difference in outcome between privileged and unprivileged groups.

As mentioned above, the Disparate Impact score is based off a legal concept in the United States to define "unintentional discrimination" and hence also provides us with a benchmark for what level of discrimination is deemed as "disproportionately adverse". Said benchmark is known as the "Four Fifths Rule", defining an adverse level of discrimination as a disparate impact score of less than 0.8 (corresponding to 80%).

A useful way to use Disparate Impact score to determine how large the potential bias is with respect to the 4/5ths Rule is to calculate the absolute value of (1 - DI) (referred to as $|1 - DI|$ later in the report). Here, a score of greater than 0.2 represents a breach of the rule, and when mitigating bias we seek to minimise this score.

Where 1 denotes a positive outcome for the decision $\hat{Y}$, and $D$ is the variable marking if an instance is privileged or unprivileged, we calculate Disparate Impact as the following:

$$DI = \frac{Pr(\hat{Y} = 1|D = \text{unprivileged})}{Pr(\hat{Y} = 1|D = \text{privileged})}$$

### 3.1.2 Statistical Parity Difference

The Statistical Parity Difference (SPD) - also sometimes referred to as the Calders-Verwer Score after being used as a metric for calculating discrimination in [5] - is a group fairness metric measured of the *difference* between the proportion of favourable outcomes for the unprivileged groups to the proportion of favourable outcomes for the privileged group. Hence, a negative score implies a benefit in favour of the privileged group and a positive score implies the opposite. An "ideal" score in this case would be 0, as this would imply a similar situation outlined above whereby there is no difference in outcome between privileged and unprivileged groups.

Whilst there is no legal benchmark attached to the SPD score in the same way as Disparate Impact, we still use the magnitude of this quantity to define how large the possible level of discrimination. For instance, a SPD score of -0.2 describes a situation

where the privileged group gets assigned favourable outcomes 20% more often than members of the unprivileged group (conversely, a score of +0.2 describes when an unprivileged group is given favourable outcomes 20% more often than the privileged group).

Where 1 denotes a positive outcome for the decision $\hat{Y}$, and $D$ is the variable marking if an instance is privileged or unprivileged we calculate Statistical Parity Difference as the following:

$$SPD = Pr(\hat{Y} = 1|D = \text{unprivileged}) - Pr(\hat{Y} = 1|D = \text{privileged})$$

During the Case Study conducted in Part 2 of this report, I will be reporting both the above score and the DI value for the test dataset, and each set of predictions output by various models both before and after the application of a range of bias mitigation tactics. This will offer some perspective of how the magnitude of group unfairness is affected with respect to the amount of unfairness latent in the test dataset itself, across a range of classification models and bias mitigation techniques.

### 3.1.3 Generalised Entropy Indices

As mentioned above, both Disparate Impact and Statistical Parity Difference only give us a picture of *group fairness*; how positive/negative outcomes are distributed through groups of people. However, there exist situations in which there can be group fairness (i.e adequate DI/SPD scores), but are still in general unfair when individual fairness is taken into account. Dwork et al. [11] describe a set of such situations where in the pursuit of group fairness between groups we end up sacrificing within-group fairness. This still represents a bias that should be diagnosed and mitigated, as within-group unfairness still presents a situation where individuals are discriminated against based upon sensitive attributes rather than individual characteristics. In [11] it is successfully argued that optimising our algorithms to maximise group fairness alone is not sufficient for determining whether or not a process is fair, and that individual fairness should be taken into account.

As a result, it is useful to include a metric whereby we can check the fairness of the outcomes of an automated decision making project with respect to *both* group and individual fairness. One such set of unified metrics for auditing both group and individual fairness are the Generalised Entropy Indices (GEI), these are metrics which have been widely used to quantify income inequality but was proposed by Speicher et al. in [25] as a means of calculating algorithmic fairness. All of the indices included in the GEI have a property known as *subgroup decompositionality*, allowing them to be decomposed into two parts representing *within-group* and *between-group* fairness. As each component is separate, there exists the possibility of being able to increase between-group fairness whilst decreasing within-group fairness. When developing automated decision making processes, it is not sufficient to only optimise group fairness as this may adversely affect the individual fairness component of the GEI. Hence, whilst a process can score well in terms of Disparate Impact or Statistical Parity, it can

still nonetheless be judged to be an unfair process when Generalised Entropy Index is taken into consideration.

Similarly to Statistical Parity Difference, the ideal value for the GEI for the application of a classifier is 0. Where an ideal score of SPD denotes a situation where a decision making process has perfect group fairness, an ideal score of any of the GEI denotes a situation where such a process is both perfectly fair to groups *and* individuals within each group. Hence, when training algorithms to make decisions, we should aim to optimise scores not only for group fairness metrics such as Disparate Impact or Statistical Parity Difference, but also for metrics which take into account individual fairness like those included in the set of Generalized Entropy Indices proposed in [25].

The formulae for generalised entropy (GE) for real values of $\alpha$ are as follows:

$$GE(\alpha) = \begin{cases} \frac{1}{n\alpha(\alpha-1)} \sum_{i=1}^{n} \left[ \left( \frac{b_i}{\mu} \right)^{\alpha} - 1 \right], & \alpha \neq 0, 1, \\ \frac{1}{n} \sum_{i=1}^{n} \frac{b_i}{\mu} \ln \frac{b_i}{\mu}, & \alpha = 1, \\ -\frac{1}{n} \sum_{i=1}^{n} \ln \frac{b_i}{\mu}, & \alpha = 0. \end{cases}$$

Within the family of Generalised Entropy indices, there is a special case of particular interest for $\alpha = 2$, which corresponds to half the squared coefficient of variation. This index is *additively decompositional*, hence it can be expressed as the sum of the between-group and within-group fairness. As a result, the GEI with $\alpha = 2$ allows us to quantify algorithmic unfairness with respect to group and individual unfairness [25].

Where $GE_B(2)$ represents the component of between-group fairness, and $GE_W(2)$ represents the component of within-group fairness. The algorithmic fairness for the entire population ($GE(2)$) can be expressed as the following:

$$GE(2) = GE_B(2) + GE_W(2)$$

Further in the report during my case study when I refer to the GEI scores reported in my results, I will be referring to the above quantity as calculated on the predictions upon a test dataset with respect to fairness. This will offer some perspective of how both group and individual fairness is affected by different classifiers and various bias mitigation tactics.

## 3.2 Feature Suppression

The most naive way to approach mitigating algorithmic bias would be to simply remove the sensitive attributes from the data during the pre-processing stage in order to prevent the model from learning correlations between such attributes and a positive or negative outcome. However, in practice this approach does not remove bias, as models often learn *indirect associations between sensitive attributes and outcome* through correlations between them and non-sensitive attributes that are left in the data.

A commonly described example of this would be as outlined by Calders & Zliobaite in [6], where removing ethnicity from a dataset whilst retaining postal code can allow models to learn to indirectly discriminate against minority ethnicities based upon location. For instance, if certain minority ethnic groups mostly live in the same set of locations, a model can learn to discriminate against this set of locations despite not directly knowing the ethnicity of the person. This effect is described as a form of automated "*red-lining*" [5] that is imposed by the algorithm, whereby a classifier learns to discriminate indirectly through the application of rules upon "*proxy*" attributes that are correlated to sensitive features.

Hence, it is not sufficient for us to simply remove sensitive features when learning models for application in automated decision making processes, as this does not suppress the possibility of discrimination. I'd also argue that given the growing need for transparency in AI (as described by Martin in [19]), we need to ensure that sensitive attributes are included when training models. Including them allows for the possibility of auditing processes, as we can directly calculate how sensitive features affect outcomes.

Additionally, Danks & London [9] suggest the pragmatic use of sensitive features can be useful, as we shouldn't pretend that correlations between them and outcomes do not exist. They argue that sensitive features contain relevant information to classifiers for learning models, and that they're not necessarily solely a vehicle for bias to be introduced. The problem with using sensitive features in automated decision processes does not come solely from including them in the data, but from allowing models to learn to discriminate (i.e apply disproportionately negative outcomes) based upon them.

During the case study following in this report, I'll investigate the effects of feature suppression upon my chosen dataset, to confirm that simply removing sensitive features from the data is not an effective means of mitigating bias.

## 3.3 Pre-processing Methods to Mitigate Bias

As introduced above, the first stage in the classification pipeline where we can attempt to mitigate bias is by *adapting the training dataset* in the pre-processing stage in order to sanitise any latent biases present within it. This prevents the model from picking up such biases, as it learns from a dataset which presents a fairer representation of the domain in which it learns from.

Below I will discuss two examples of pre-processing algorithms for bias mitigation, before applying them to an existing dataset within my case study in Part 2 of this report. I'll then attempt to offer some perspective on how they perform relative to classifiers trained on the original dataset with respect to performance and the fairness metrics discussed above.

### 3.3.1 Disparate Impact Remover

A pre-processing algorithm proposed by Feldman et al. in [12], it aims to "*repair*" a dataset in order to remove Disparate Impact whilst retaining rank-order within groups. In the context of this algorithm, to "repair" a dataset is to attempt to construct an augmented version of the original dataset. One such that by changing some of the labels within it, that any potential level of disparate impact with respect to sensitive attributes has been minimised/removed and group fairness is achieved.

Hence, the goal of the Disparate Impact Remover algorithm is to produce a training dataset $D'$ which is derived from the original training dataset $D$; whereby the Disparate Impact is minimised with respect to Sensitive Attribute $X$. Following the construction of $D'$, you simply train your model as usual and test it using an un-augmented test set.

### 3.3.2 Reweighing

This is a pre-processing algorithm introduced by Kamiran & Calders in [14], however unlike the Disparate Impact Remover it does not change any labels within the original dataset. Instead, it adds a *weight parameter* to each instance in the dataset which corresponds to both the outcome observed for the instance and whether or not it belongs to a protected group. So, for instance a member of a privileged group with an associated favourable outcome is assigned a lower weight than a member of an unprivileged group with a favourable outcome. The reweighing algorithm seeks to attain group fairness with respect to Statistical Parity Difference, by minimising this quantity through the process of assigning higher weight to positive instances of unprivileged groups and lower weight to corresponding instances from the privileged groups.

The weights for each instance in the dataset is calculated as the ratio of the *expected* probability of a positiveoutcome for an unprivileged group member and the *observed* probability value. As shown below, where $W(X)$ represents the weight value for instance $X$, $\Pr_{exp}(S = unprivileged \wedge class = +)$ represents the expected probability of a positive outcome for an unprivileged instance, and $\Pr_{obs}(S = unprivileged \wedge class = +)$ represents the corresponding observed probability, the weight value is calculated as:

$$W(X) = \frac{\Pr_{exp}(S = unprivileged \wedge class = +)}{\Pr_{obs}(S = unprivileged \wedge class = +)}$$

Once a set of weights has been calculated for each instance in the dataset, as with the output of the Disparate Impact Remover, you train a model as usual on the output of the Reweighing algorithm and test on an un-reweighed test set.

A potentially interesting fact to note here is that the above pre-processing algorithms attempt to optimise for each of the two group fairness metrics discussed in Section 4.1. Where the Disparate Impact Remover algorithm attempts to minimise the Disparate Impact value for the training dataset, and the Reweighing algorithm attempts to minimise Statistical Parity Difference.

## 3.4 In-processing Methods to Mitigate Bias

The next stage in the pipeline where we can attempt to mitigate bias is by using a model which is itself "*fairness aware*". This means that during training it can self-audit for fairness by adjusting a set of parameters to minimise potential bias as it learns to produce a fairer model.

In the following section, I will discuss a method of in-process bias mitigation known as Adversarial Debiasing, before applying it to an existing dataset within my case study in Part 2. Similar to the pre-processing algorithms discussed above, I will offer some perspective on how the model produced via Adversarial Debiasing performs relative to other classifiers with no bias mitigation techniques applied, with respect to performance and fairness.

### 3.4.1 Adversarial Debiasing

A supervised, deep learning bias mitigation algorithm proposed by Zhang et al. in [30], whereby the model attempts to simultaneously learn a classifer to maximise prediction accuracy and reduce an adversary's ability to determine the protected attribute from the predictions. As the predictions do not carry any group discrimination information, such that an adversary could exploit, it can be said that Adversarial Debiasing produces a fair classifier.

By supplying the model with the data and a variable denoting the unprivileged group of interest, the Adversarial Debiasing algorithm - as alluded to above - simultaneously learns a predictor and an adversary. While the predictor seeks to optimise with respect to classification accuracy, the adversary attempts to model the unprivileged group supplied. Hence, when training the Adversarial Debiasing model, we attempt to maximise its ability to predict an outcome accurately and minimise an adversary's ability to predict the value for the supplied protected attribute (for example race, or gender).

## 3.5 Post-processing Methods to Mitigate Bias

The final stage in the pipeline we attempt to mitigate bias is by changing the *outcomes* predicted by the model with respect to fairness. Given some idea of what a fair representation should look like, we can audit the output of a model and attempt to make adjustments to these outputs relative to such a view and produce a more fair set of predictions.

Here, I will discuss two examples of post-processing algorithms for bias mitigation, before applying them to an existing dataset within Part 2. As with the pre and in-processing bias mitigation tactics, I will offer a similar perspective toward the performance and fairness of post-processing algorithms compared to the outputs of classifiers with no bias mitigation techniques applied.

### 3.5.1 Calibrated Equalised Odds Postprocessing

This is a post-processing bias mitigation algorithm proposed by Pleiss et al. in [22], whereby the classification outputs from a model are adjusted with respect to calibrated probability estimates for the privileged and unprivileged groups and an equalised odds objective.

The Equalised Odds algorithm as described by Hardt et al. in [13] seeks to ensure that no error rate disproportionately affects either the privileged or unprivileged groups. Both groups should have both the same False Positive Rate and False Negative Rate. However, the Calibrated Equalised Odds (CEO) algorithm adds the constraint of also maintaining probability estimates for positive/negative outcomes for both groups on top of the equalised odds constraint.

Unfortunately, the CEO algorithm could not produce satisfactory results when the entire equalised odds constraint is applied (i.e didn't perform as necessary when maintaining both error rates), however it can whenever only considering the False Positive Rate/False Negative Rate alone, or a weighted combination of the two.

### 3.5.2 Reject Option Classification

A post-processing bias mitigation algorithm proposed by Kamiran et al. in [15], whereby it reassigns favourable outcomes to members of unprivileged groups and unfavourable outcomes to members of privileged groups in a "*confidence band*" denoting a region around the decision boundary with the highest uncertainty for each group.

Reject Option Classification (ROC) attempts to utilise the posterior probabilites produced by classifiers to identify instances for relabelling in a manner that neutralises discrimination. Probabilistic classifiers use posterior probabilities to assign predictions to instances - for example $Pr(\hat{Y} = 1|D = \text{unprivileged})$ represents the probability of an instance being assigned to class 1 given that such instance is a member of the unprivileged group. If this probability value is close to 1.0 or 0.0 then it can be said that the classifier has a high level of confidence in its prediction. However, if the value is closer to 0.5, then the classifier is more uncertain. The ROC algorithm focuses on reducing discrimination by changing the labels for such instances, where the classifier doesn't have an adequate level of certainty. This area in the decision space is known as the *critical region*, and is considered to contain the instances where the classifier is most likely to be subject to bias.

Hence, the ROC algorithm initially leaves all instances in the critical region as unlabelled (known in this context as *rejected*). After rejection, ROC will check to see if the instance is marked as a member of the privileged or unprivileged group. If the instance is a member of the privileged group, ROC will assign a negative outcome to it. Whereas if the instance is a member of the unprivileged group, ROC will assign a positive outcome to it.

# Part II

# A Case Study: How to Quantify and Mitigate Algorithmic Bias

# Chapter 4

# Methods

## 4.1 The Dataset Used

I have chosen a dataset reporting the Three-year Recidivism for Offenders Released from Prison in the State of Iowa. This is a useful context to study, given attempts in the past to obtain accurate models to classify offenders by their likelihood to re-offend. Such models have been shown to exhibit a clear bias against members of minority ethnic groups and a bias against males, particularly in the US. In recent years, ProPublica produced a report that exposed a significant amount of bias against Black Males in the application of an offender risk prediction algorithm known as COMPAS, which can be found at [2]. Therefore, it would be useful to see if applying some of the techniques discussed above along the classification pipeline can mitigate any biases propagated through training or amplified at the training stage for a range of classifiers.

Whilst other states in the US have open data for analysing recidivism rates, they often did not offer enough context for me to train a model in a meaningful way by either not including data on the offender's background, their offences, or both. In contrast, the State of Iowa's dataset included the above attributes, allowing for more contextual information to be used when training a model upon it. Additionally, the size of the dataset is modest (21646 entries), such that it is large enough that I can train a model that will hopefully also help give a meaningful outcome.

As you can see in Table 4.1 overleaf, the data covers a wide set of attributes that includes some of particular interest for my case study, including:

- Race/Ethnicity
- Sex
- Age At Release
- Convicting Offense Classification, Type & Sub-type
- Release Type
- Main Supervising District

- Year Released

I can use the above set of attributes for each instance in the dataset to train a set of binary classification algorithms, in order to produce a set of models for predicting recidivism. I will then provide an analysis of the performance/fairness metrics calculated for each classifier before any mitigation algorithms are employed, and after I have applied them at each stage of the pipeline. Additionally, I will confirm that simply suppressing sensitive features by removing them from the training data is not a sufficient way to resolve the problem of algorithmic bias.

## 4.2 Pre-processing Steps Applied to Data

Before being able to train a model upon the datasets chosen for this case study, some pre-processing was required in order to get the data into a suitable form.

Firstly, I loaded the CSV file containing the entire dataset into a Pandas DataFrame object in order to visualise all the dimensions available, their form, how many categories for each attribute existed and the distribution of their values.

Next I reformatted all the columns and entries in the dataset in order to get a consistent representation and make values more predictably searchable, and easy to understand. I simply removed special characters, replaced spaces with underscores and lower cased all the strings in the data across the column names and the row entries. At this point I also mapped the string values for the recidivism attribute to an integer value, classifying all rows with ”yes” as 1 and all rows with ”no” as 0. Hence, a ”*favourable outcome*” in the context of classification of the data will be a 0, i.e a prediction that the instance associated with this offender will not re-offend.

### 4.2.1 Feature Identification

Above I have identified a set of features within the data that were of interest. Within the dataset there were also a set of attributes that would only be known for *positive* instances within the data (i.e Days to Recidivism, Recidivism Type). So the feature identification stage of pre-processing the data simply involved filtering out the attributes corresponding to only positive instances within the data and restricting it to the set of interesting features I have described above. These attributes are related to a set of sensitive attributes, race/ethnicity, sex, and age, as well as a set of attributes related to the person’s original offence, the year they were released and what kind of release it was (i.e paroled or discharged).

Table 4.1: Attributes available in the 3-Year Recidivism for Offenders Released from Prison in Iowa dataset.

| Attribute Name | Description | Attribute Type |
|---|---|---|
| Fiscal Year Released | Fiscal year for which the offender was released from prison | Integer (categorical) |
| Recidivism Reporting Year | Fiscal year that marking the end of the 3-year tracking period | Integer (categorical) |
| Race - Ethnicity | Offender's Race and Ethnicity | Plain Text (categorical) |
| Sex | Offender's Gender | Plain Text (categorical) |
| Age At Release | Offender's age group at release from prison. | Plain Text (categorical) |
| Convicting Offense Classification | Maximum penalties:<br>• A Felony = Life<br>• B Felony = 25 or 50 years<br>• C Felony = 10 years<br>• D Felony = 5 years<br>• Aggravated Misdemeanor = 2 years<br>• Serious Misdemeanor = 1 year<br>• Simple Misdemeanor = 30 days | Plain Text (categorical) |
| Convicting Offense Type | General category for the most serious offense for which the offender was placed in prison. | Plain Text (categorical) |
| Convicting Offense Sub-type | Further classification of the most serious offense for which the offender was placed in prison. | Plain Text (categorical) |
| Release Type | Reasoning for Offender's release from prison. | Plain Text (categorical) |
| Main Supervising District | The Judicial District supervising the offender for the longest time during the tracking period. | Plain Text (categorical) |
| Recidivism - Return to Prison | • No = No Recidivism<br>• Yes = Prison admission for any reason within the 3-year tracking period | Plain Text (categorical) |
| Recidivism Type | • New = Admission to prison with a new conviction<br>• Tech = Admission to prison for a violation of supervision requirements (Technical reasons) | Plain Text (categorical) |
| Days to Recidivism | Number of days it took before the offender returned to prison. | Integer (continuous) |
| New Conviction Offense Classification | New conviction maximum penalties | Plain Text (categorical) |
| New Conviction Offense Type | General category for the new conviction while the offender is out of prison. | Plain Text (categorical) |
| New Conviction Offense Sub Type | Further classification of the new conviction | Plain Text (categorical) |
| Part of Target Population | The Department of Corrections has undertaken specific strategies to reduce recidivism rates for prisoners who are on parole | Plain Text (categorical) |

### 4.2.2 Handling Null Values

I then checked to see if any null values existed for the features that I had identified to train the classifiers upon. Of the 9 attributes I chose to use, 5 of these had rows with

null values. These rows and their total of missing values out of a total of 21646 were:

- Race - Ethnicity: 30
- Sex: 3
- Age at Release: 3
- Release Type: 1762
- Main Supervising District: 8470

For the race/ethnicity, sex, age and release type missing values I decided to map the null values to their most respective most common values. For Race/Ethnicity this was ”White”, Sex this was ”Male”, Age at Release this was ”25-34” and Release Type this was ”Parole”.

As for the main supervising district attribute, I discovered that all the null values in this column corresponded to individuals who had been discharged (as opposed to paroled in any form). I.e only those who had been paroled had been allocated a main supervising district. Hence, I decided to map all the null values in this column to ”NA” (not applicable). At this point I checked the Release Type and Main Supervising District columns to see if there were any instances where I’d be assigning ”parole” to someone without a corresponding main supervising district (i.e parole to someone who was in fact discharged) and found no such occurrences.

### 4.2.3 Reducing Data with Low Value Counts

At this point I sought to reduce the amount of low occurrence values within the data, as there were a lot of inconsistencies with respect to how data was entered into the database and for some attributes a lot of unnecessary subdivision of categories.

Firstly, I had a look at the Race/Ethnicity attribute, as I had discovered that the Hispanic population hadn’t its own value but was instead encoded with respect to other races. For instance ”White - Hispanic”, ”Black - Hispanic”, I decided to remap such values to just ”Hispanic” alone and to consolidate the Hispanic population within the data.

Next, I looked at the Release Type attribute and saw there was much unnecessary subdivision, inconsistency and low-occurring variations of release types that could be removed by remapping to a standard set of categories: Parole, Discharged, Special Sentence. For instance, there were 8 variations of Parole incident in the data of which 2 substantially identical types accounted for 96% of individuals who were paroled. As a result, I remapped any instance of Parole to the same key. Both types of Discharge encoded in the data were substantially the same, so I remapped any instance of Discharge to the same key. Similarly, both types of Special Sentence encoded were substantially the same and were remapped to the same key.

Following this was some similar work to reduce the Convicting Offence Classifications within the data; as this attribute also suffered from some of the same issues as the Release Type attribute. For instance, there were a few low-occurring subdivisions of

Felony charges as well as an encoded ”other felony” category, hence I mapped all such occurrences to the same key. There were two substantially similar classifications encoded for sexual offences that I also remapped to the same key.

### 4.2.4 One-Hot Encoding

At this point I was ready to transform the dataset into a form that I could run train a set of classifiers on. As the data was all categorical, with no continuous variables, I decided that the most suitable way to encode the data was using One-Hot Encoding to produce a binary representation of each instance within the data. A small example of what One-Hot Encoding a dimension of data looks like can be found in Figure 4.1.




Figure 4.1: An example of One-Hot Encoding applied to a set of values for the Sex attribute

I considered the possibility of encoding the string labels as integer values as this would prevent the dimensionality of the dataset blowing up. However, I decided against this as none of the attributes were ordinal, i.e there was no way of defining a discrete ordering of the columns. For instance, if I decided to label encode the Race/Ethnicity attribute such that White was 1 and Black was 2 (or vice versa), this would introduce the implication that a value for Black was twice the value for White (or vice versa), which didn't make a lot of sense in this context.

As a result, the dimensionality of my attributes increased vastly up to a total of 72 dimensions, plus the class variable. However, the previous step of reducing the number of categories for some of the attributes within the data curtailed this by some amount.

### 4.2.5 Handing Imbalanced Classes using SMOTE

As the dataset showed an imbalance towards the negative class (i.e those who did not reoffend), I decided to apply the Synthetic Minority Over-sampling Technique (*SMOTE*) [7] to the training set. This results in a balanced input when training each of the models employed, by over-sampling the negative class using synthetic data-points which are *derived from real instances* within the dataset.

As opposed to other oversampling techniques, whereby we replace majority examples with minority ones, SMOTE generates synthetic examples of members of the minor-

ity class. For each member of the minority class, it looks at it nearest neighbour and subtracts it from the current subject before multiplying this difference by a random number between 0 and 1 and adding that quantity to the subject. This selects a random point between the two neighbours and assigns it to the minority class, making it more general, i.e making the decision boundary for those who reoffend less specific. This resulted in the set of models trained discriminating between positive and negative instances a lot better and reducing their False Positive rates.

As you can see in Table 4.2, before SMOTE is applied the training data set has a size of 15152 instances, with an imbalanced split of 4929 instances in the negative class (i.e marked as reoffenders) and 10223 in the positive class. After SMOTE is applied, the training set is expanded to 20446 instances, with an equal split of 10223 instances in each class.

Table 4.2: Below shows the effects that SMOTE has on training data, oversampling the minority class in order to create a balanced set.

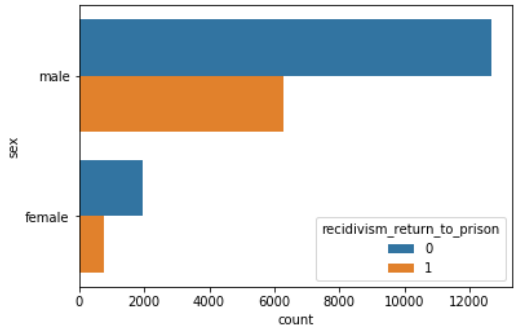
| | **Counts of label: 1** | **Counts of label: 0** |
|---|---|---|
| **Pre-SMOTE** | 4929 | 10223 |
| **Post-SMOTE** | 10223 | 10223 |

## 4.3 Data Exploration

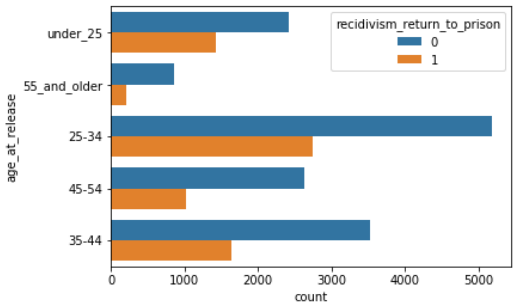
After having pre-processed the data, I conducted some exploration into how recidivism is distributed throughout each of the attributes above that I sought to train my classifiers on. I thought it would be useful to be able to visualise what kind of trends existed in the data itself, the countplots I produced can be seen in Figure 4.2 and a further countplot related to ”Main Supervising District” is found in Figure 4.3c. I also had a look at how race/ethnicity of offenders - particularly White & Black offenders - was distributed throughout the various ”Main Supervising Districts”, as I felt it would be interesting to see if this attribute had the potential to become a *proxy variable* for a person’s race/ethnicity. These can be seen in Figures 4.3a, and 4.3b.

Looking at the countplots in Figure 4.2 gave me the following insights:

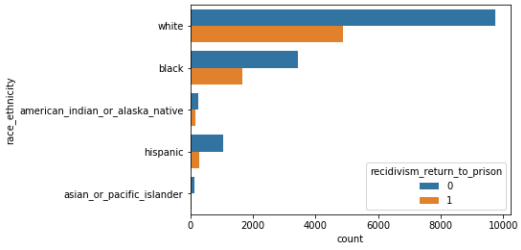
- The vast majority of instances in the data are male offenders, additionally the proportion of males that reoffend is larger than the proportion of females that reoffend.
- Around 95% of the instances in the data are white or black, the vast majority of which are white. Proportionately, both whites and blacks are roughly equal in terms of recidivism rate.
- Class C and Class D felonies are the most common offence classifications, followed by Aggravated Misdemeanour. There are also many offence classification categories with very low occurences.



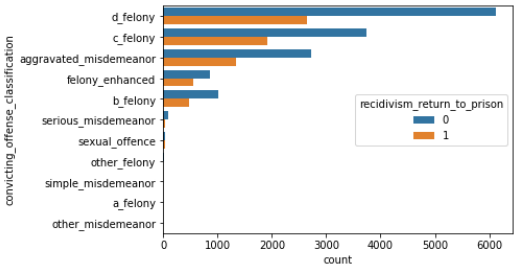

(a) Recidivism vs. No Recidivism for Each Sex



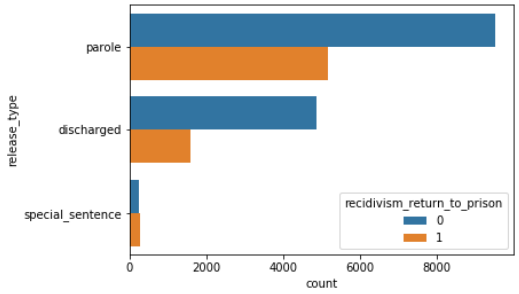

(b) Recidivism vs. No Recidivism for Each Age Group



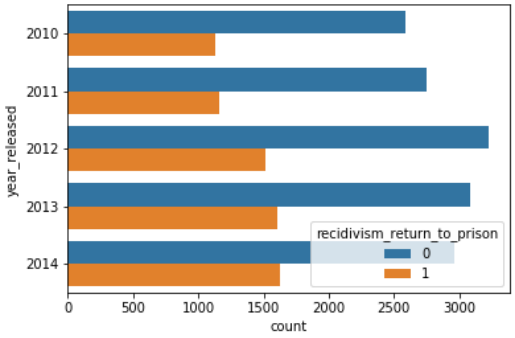

(c) Recidivism vs. No Recidivism for Each Race




(d) Recidivism vs. No Recidivism for Each Offence Classification



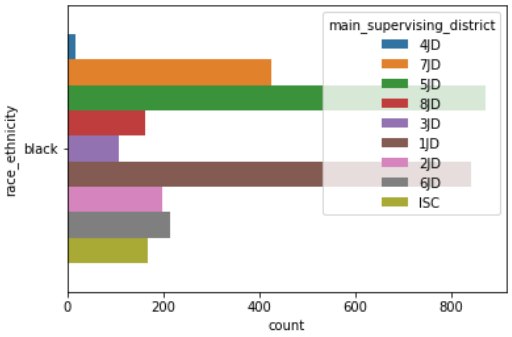

(e) Recidivism vs. No Recidivism for Each Release Type



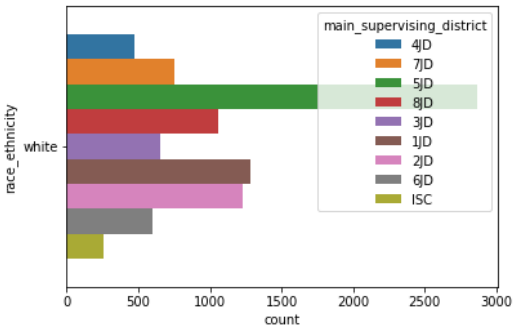

(f) Recidivism vs. No Recidivism for Each Release Year

Figure 4.2: Countplots showing distribution of recidivism for a range of attributes

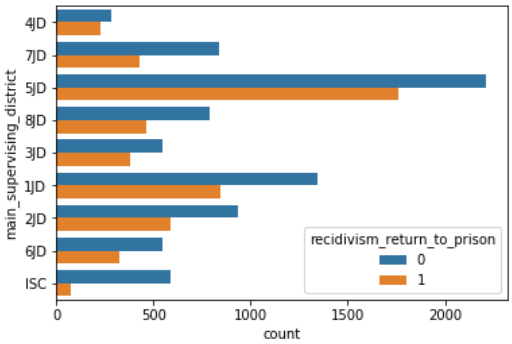
- Although majority of offences are classified as D felonies, the proportion of these instances that reoffend is lower than Class B and C Felonies, Aggravated Misdemeanour, and Felony Enhanced.
- The majority of offenders are released under Parole, and paroled offenders are much more likely to reoffend than those who are discharged.
- While there weren't many instances of Special Sentences in the data, the majority of these instances actually represented reoffenders.
- Between 2010 and 2012, there was an increase year-on-year in prison releases, after which the number of prisoners released per year has steadied.




(a) Distribution of Main Supervising District for Black Offenders




(b) Distribution of Main Supervising District for White Offenders




(c) Recidivism vs. No Recidivism for Each Main Supervising District

Figure 4.3: Countplots showing distribution of race across Main Supervising Districts, and their respective Recidivism Rates

- Year-on-year, the proportion of prisoners released who go on to reoffend increases.

Additionally, the countplots shown in Figure 4.3 offered the following:

- There seems to be some variation in where white and black offenders are located, indicated by different peaks in main supervising district.
- Majority of both black and white offenders are associated with 5JD, with higher proportions also in 3JD, 2JD, 8JD, and 4JD
- White offenders seem to be more spread across supervising districts, whilst black offenders are concentrated in 5JD, 7JD, and 1JD.
- 5JD is overwhelmingly most common Main Supervising District observed in data, with a high proportion of reoffenders associated.
- While ISC represents a small population within the data, its associated proportion of reoffenders is much lower than the rest of the categories.

## 4.4 Scikit-learn

Python library offering a wide range of useful tools for Data Analysis & Machine Learning, described by its developers in [21]. It provides developers/scientists/data professionals with a low-dependency (only requiring Scipy and NumPy), and well-documented interface for building Machine Learning and Statistical Data Analysis Systems.

As it included implementations of many popular/widely-used classification algorithms, a wide set of performance metrics, means of conducting cross-validation, and means of tuning parameters for the application of classification algorithms; Scikit-learn allowed me to kickstart my project a lot more easily by cutting down time spent implementing, training, and testing models upon my dataset.

### 4.4.1 Models Used

I decided for this project it would be suitable for me to test how the bias mitigation techniques discussed in Part 1 would perform across a range of classification algorithms, offering more context to the reader as to what tactics suit various situations. For instance, perhaps one technique (i.e pre-processing bias mitigation) may perform well in one context but not generalise across a range of contexts. Hence, I selected the following set of classification algorithms, which I will provide a short introduction to below.

#### 4.4.1.1 Logistic Regression

The basic principle of Logistic Regression classification is to use the Sigmoid Function (also known as Logistic Function, shown in Figure 4.4) to model a binary dependent variable. Such that if $\Pr(\hat{Y} = 1|X) > 0.5$ where $\hat{Y} = 1$ denotes a positive classification and $X$ denotes the instance under classification, then $X$ is classified as positive and otherwise is classified as $(\hat{Y} = 0)$. In Logistic Regression, the above probability is calculated via the sigmoid function, converting a vector to a value between 0 and 1 based on a set of weights calculated during the training stage.

While logistic regression is typically very efficient, and provides a good baseline, one main disadvantage it suffers from is that it doesn't perform well with respect to datasets that are not linearly separable.

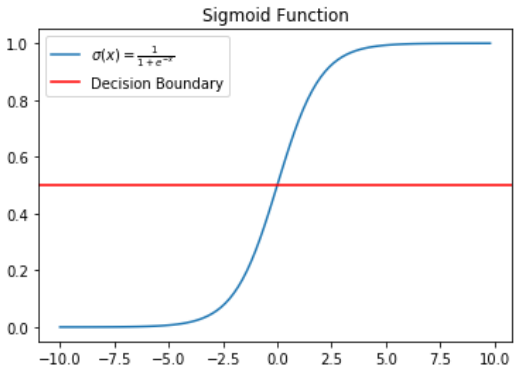



Figure 4.4: A plot of the Sigmoid Function, used in Linear Regression to classify instances.

#### 4.4.1.2 Random Forest Classifier

This is an *ensemble* method, whereby a multitude (i.e a *forest*) of Decision Trees are built from random subsets of the data supplied during the training stage. When classifying instances, the Random Forest Classifier simply outputs the average prediction made by the set of Decision Trees. Such types of ensemble methods are known as "*bagging*" methods, as it gets its output from taking the average of a set of independent classifiers.

Similarly to using a lone decision tree, this method can be subject to overfitting, however this can easily be remedied by ensuring to use a large enough number of trees in the forest as it will make the predictions more generalised.

#### 4.4.1.3 Gradient Boosting Classifier

Similarly to the Random Forest Classifier, this is another example of an ensemble method. However, instead of simply taking the average output and assigning instances to it, it produces a final model *iteratively* by adjusting parameters using *gradient descent* to find an optimal value for a defined *loss function* such as Mean Squared Error or the Logistic Function. This type of ensemble method is known as a "*boosting*" method, whereby it makes its predictions sequentially as the classifier converges over each iteration. The name Gradient Boosting itself comes from its combination of using the Gradient Descent algorithm and Boosting.

As with the Random Forest Classifier, Gradient Boosting classifiers can be prone to overfitting and often requires fine-tuning of parameters to combat this and optimise performance.

#### 4.4.1.4 AdaBoost Classifier

AdaBoost is another example of an ensemble method which uses boosting to predict a model, short for "*Adaptive Boosting*". It can be said that AdaBoost represents a special

case of Gradient Boosting, as its mechanism for making predictions is the same. The difference being that AdaBoost uses a particular loss function for its Gradient Descent, whereas Gradient Boosting is more general and can use a multitude of different loss functions. Specifically, AdaBoost uses the Exponential Loss function.

Often used as a baseline classifier due to generally high performance, it is less prone to overfitting than alternatives. However it can be sensitive to noisy data and the presence of outliers in the data.

#### 4.4.1.5 Multi-Layer Perceptron

The Multi-Layer Perceptron (MLP) is a class of "*feed-forward neural network*", composed of at least three layers of nodes: the input layer, one or more hidden layers, and an output layer. Each layer that follows the input is set of neurons that apply a non-linear "*activation function*" such as the Logistic Function, or Rectified Linear Unit (ReLU) Function (shown in Figure 4.5).

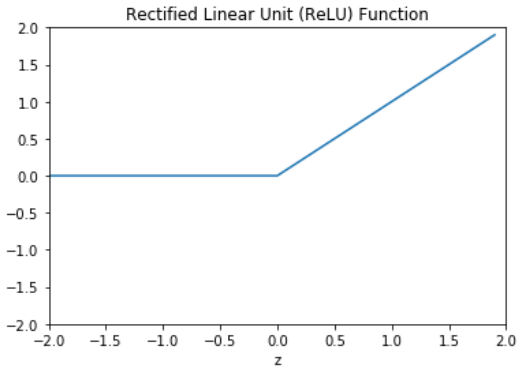



Figure 4.5: A plot of the ReLU Function, commonly used as an activation function by Multi-Layer Perceptrons and other Artificial Neural Networks.

Multi-Layer Perceptrons are often used in cases where it is necessary to classify data that is not linearly separable, and particularly for classifying tabular datasets such as the one I am using. For other more complex workloads, such as image classification, the MLP has been superseded by Convolutional Neural Networks and other Deep Learning algorithms.

## 4.5 AI Fairness 360

Another Python library I used extensively was AI Fairness 360, an open-source library and algoritmic fairness project started by IBM. It seeks to provide developers with a range of tools to quantify and mitigate bias in Machine Learning models, and is described by its developers in [4]. Including implementations for a range of metrics and bias mitigation algorithms, including those introduced earlier in the report. This allowed me to easily benchmark the performance of each tactic with respect to the

metrics discussed along all three stages of the fairness pipeline as described by [4] and shown in Figure 4.6.

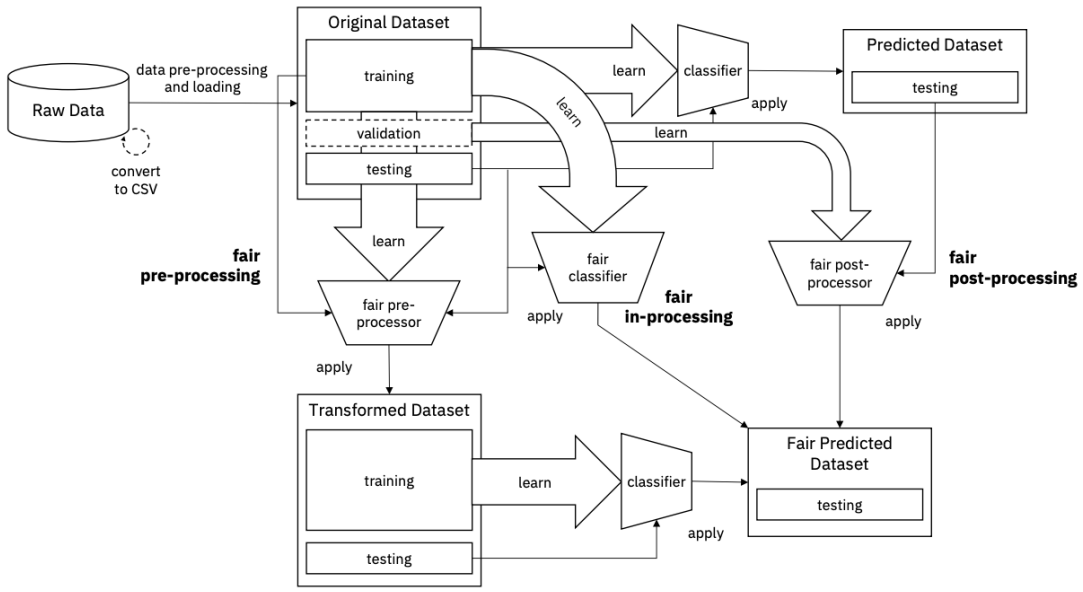



Figure 4.6: Fairness Pipeline as described in [4], showing 3 pathways to fairer predictions. Arrows marked "Learn" correspond to the `fit` method for the algorithm being called, and those marked "Apply" correspond to the `transform` or `predict` method being called.

The main goals as described by the developers include the facilitation of the transition from researching algorithmic fairness to wide use in an industrial setting, by providing a common framework for researchers to share and evaluate algorithms. I.e researchers can contribute their algorithms/metrics directly to the project, as it is open source. Additionally, they seek to promote understanding of algorithmic fairness, encouraging and enabling researchers to share and benchmark their work as well as developers/data professionals to apply these principles in their work.

A UML class diagram, and a set of code snippets can be found in the appendices of [4]. This can be used to reference how the classes interact with each other, and gain some understanding of the programming paradigm used by AIF360 to produce and evaluate fairer classifications.

The system itself is designed in an extensible way, in order to achieve the goal of encouraging contribution. As it defines a set of four generalised and well-documented abstractions for developers to interact with. These include:

- The `Dataset` Class, which defines a common interface/format for interacting with any/all data used and contains a set of subclasses to handle different data structures.
- The `Metrics` Class, providing the standard interface for obtaining a wide range of group and individual fairness metrics upon both the original dataset and the output from classifications. It also includes a range of performance-related met-

rics that can be calculated with respect to classification outputs compared to the ground truth.

- The `Explainer` Class, which provides the user an interface to obtain further insights into computed fairness metrics. The two subclasses of which are `TextExplainer` and `JSONExplainer`, providing explanations in either a plain text or JSON format.
- The `Algorithms` Class, including subclasses containing interfaces for utilising pre-, in-, and post-processing bias mitigation algorithms. The user can choose which ever stage in the pipeline that suits them or benchmark all three, accordingly.

Hence, if the user would like to contribute to the project, they translate their implementation of the bias quantification/mitigation tactic they would like to add into one that adheres to the AIF360 class structure. As the library has extensive documentation available, the user can leverage that when doing so.

# Chapter 5

# Outcomes

Table 5.1: Below shows the fairness metrics discussed in Part 1 as calculated on the original test dataset predicted by the models learned.

| ***Fairness Metrics Computed on Test Dataset*** | | |
|---|---|---|
| **DI** | **abs(1-DI)** | **SPD** |
| 0.921 | 0.079 | -0.058 |

After splitting my dataset up into a 70/30 train-test split, my first step before applying any bias mitigation techniques was to find out how ”fair” the test set itself was with respect to sex. I wanted to see what magnitude of bias could be latent within the data, and also use these values to compare with the fairness metrics computed on the classification outputs from the various models (both pre and post bias mitigation applications). As the classifications could either improve upon, or exaggerate any level of bias that occurs within the data itself.

The *protected attribute* I chose to examine here was Sex. As I believed that the difference in proportion of those who reoffend between male and female, on top of the large difference in the number of instances between the two, would result in any ”naive” classifier (i.e one for which no bias mitigation techniques are applied) learning a bias against males.

As you can see in Table 5.1, there was a very slight amount of difference in outcome against males. With a Disparate Impact score of 0.921 (and 0.079 for the respective $|1-DI|$ value), and a Statistical Parity Difference score of -0.058. The SPD score shows that there is a corresponding 5.8% difference in favour of females in the proportion of favourably labelled data. I.e males were proportionately 5.8% more likely to reoffend in the real world. It can be said that the true data doesn't seem to possess any appreciable margin of difference in outcome between the sexes, but when training a classifier there is the possibility that it can exaggerate the small quantity of difference of outcome that exists.

## 5.1 Baseline Performance & Fairness of Models

Table 5.2: Metrics as calculated on the test dataset predictions before any bias mitigation tactics were applied.

| | **Performance Metrics** | | | **Fairness Metrics** | | | |
|---|---|---|---|---|---|---|---|
| **Model** | Accuracy | Precision | AUC ROC | DI | abs(1-DI) | SPD | GEI |
| RFC | 0.70 (+/- 0.06) | 0.69 (+/- 0.03) | 0.76 (+/- 0.08) | 0.874 | 0.126 | -0.092 | 0.204 |
| LR | 0.68 (+/- 0.12) | 0.69 (+/- 0.08) | 0.76 (+/- 0.14) | 0.752 | 0.248 | -0.231 | 0.157 |
| MLP | 0.69 (+/- 0.10) | 0.68 (+/- 0.06) | 0.77 (+/- 0.13) | 0.912 | 0.088 | -0.065 | 0.190 |
| GB | 0.65 (+/- 0.05) | 0.63 (+/- 0.03) | 0.71 (+/- 0.06) | 0.738 | 0.262 | -0.185 | 0.251 |
| AB | 0.64 (+/- 0.05) | 0.63 (+/- 0.03) | 0.70 (+/- 0.07) | 0.597 | 0.403 | -0.340 | 0.255 |

After splitting the dataset into a training and test set, the next step was to train instances of the models introduced above in Chapter 5. This step was conducted in order to get a baseline for the performance and fairness metrics upon the *naive* models.

When training each model, I applied 10-fold cross-validation to prevent overfitting affecting the performance metrics reported. I also applied cross-validation for each model in order to find more optimal parameters for training. Each cross-validation was conducted to obtain the mean and standard deviation scores (as shown in Table 5.2) of the each model's Accuracy, Precision, and the Area Under each ROC Curve. While the accuracy gives a picture of the proportion of predictions that were correct, it doesn't give any insight to the breakdown with respect to metrics such as False Positive Rate. For this reason, I decided to also add Precision and AUC ROC Curve metrics when considering the performance of each model.

Precision [16] - otherwise known as Positive Predictive Value $PPV$ - is defined as the following quantity:

$$PPV = \frac{TP}{TP + FP}$$

Where $TP$ represents the quantity of True Positives, and $FP$ represents False Positives. The precision value offered me a better idea of the quality of predictions made, as for all the positives predicted it shows the proportion that were actually positive (i.e the accuracy of predictions for instances classified as reoffenders).

The Area Under the ROC Curve [20] for each model gives an idea of how well the model is able to separate the positive and negative classes. Typically, this takes a value of between 0.5 and 1.0. With an ideal value of 1.0 as this implies the model can perfectly separate the classes, and a value of 0.5 being the worst possible as this implies that the model has no predictive power and effectively makes random guesses. The value otherwise corresponds to the model's ability to distinguish between classes. For instance, a value of 0.7 means that there is a 70% chance of the model correctly distinguishing between classes.

After training each model, I obtained a set of predictions for the held-out test set. The values for the reported Fairness Metrics in Table 5.2 display the Disparate Impact (and corresponding $|1 - DI|$), Statistical Parity Difference, and Group Entropy Index

scores for each of the models predictions when compared to the ground truth of the test dataset.

Comparing the Fairness Metrics in Table 5.2 for the naive models to those observed from the data in Table 5.1, you can see that every model with exception to the Random Forest Classifier and Multi-Layer Perceptron picked up a distinct amount of bias. Each of the Logistic Regression, Gradient Boosting, and AdaBoost models break the "4/5ths Rule" with respect to Disparate Impact as each of their respective $|1-DI|$) scores were greater than 0.2. While both the RFC and MLP classifiers picked up some bias, as their Fairness Metrics scores were below those in the test data, they both adhered to the 4/5ths Rule and their changes in SPD scores were minimal (3.4% and 0.7%, respectively).

As the Group Entropy Index score is a Classification Metric and hence depends on a comparison between predictions and truth, it cannot be computed upon the test dataset itself. So there is little context at this point to be able to draw any conclusions about the respective GEI scores before bias mitigation has been applied. What can be said is that there isn't a large difference between the highest (0.255 for AB clasifier) and the lowest (0.157 for LR classifier). These scores will become more relevant in the context of the application of bias mitigation, as it will be interesting to see how this value changes since it is a "hybrid" metric that takes into account both group *and* individual fairness.

Additionally, when looking at Performance Metrics further in the case study, the values seen in Table 5.2 will serve as a baseline. It will be interesting to see the impact on performance that is observed when applying various bias mitigation techniques.

## 5.2 Feature Suppression

Table 5.3: Metrics as calculated on the test dataset predictions where all sensitive features suppressed.

| ***Results from Suppressing Sensitive Features*** | | | | | | | |
|---|---|---|---|---|---|---|---|
| | **Performance Metrics** | | | **Fairness Metrics** | | | |
| **Model** | Accuracy | Precision | AUC ROC | DI | abs(1-DI) | SPD | GEI |
| RFC | 0.64 (+/- 0.03) | 0.64 (+/- 0.03) | 0.70 (+/- 0.06) | 1.145 | 0.145 | 0.076 | 0.231 |
| LR | 0.61 (+/- 0.02) | 0.61 (+/- 0.02) | 0.66 (+/- 0.04) | 1.248 | 0.248 | 0.111 | 0.256 |
| MLP | 0.64 (+/- 0.04) | 0.64 (+/- 0.03) | 0.70 (+/- 0.06) | 1.108 | 0.108 | 0.058 | 0.238 |
| GB | 0.60 (+/- 0.01) | 0.59 (+/- 0.01) | 0.64 (+/- 0.01) | 1.375 | 0.375 | 0.144 | 0.279 |
| AB | 0.60 (+/- 0.01) | 0.59 (+/- 0.01) | 0.63 (+/- 0.02) | 1.361 | 0.361 | 0.138 | 0.284 |

The next experiment I ran was to confirm what I saw frequently in the literature: that simply removing sensitive features from the dataset would not result in unbiased models. Hence, I removed the Race/Ethnicity, Sex, and Age attributes from the training/testing datasets and trained an instance of each of the models chosen for this case study.

As you can see from Table 5.3, Feature Suppression did not remove bias from the predictions made by each model. In fact, four of the five models' $|1-DI|$ scores were

equal to or worse than those observed with data including sensitive features - the only exception being the AdaBoost classifier. Additionally, as seen by the DI and SPD scores being greater than 1.0 and 0.0 respectively across all five models, the direction of the bias actually reversed when removing sensitive features. Meaning that when sensitive features in the data were removed, every model learned a bias in favour of males. Comparing Group Entropy Index scores for the Feature Suppressed and non-Feature Suppressed datasets, all five models scored higher with suppressed sensitive features. Meaning that suppressed features potentially results in an increase in both group and individual unfairness.

Looking at how performance is affected, suppressing sensitive features resulted in decreased performance across all models and all metrics. With accuracy decreasing between 5% to 8%, precision decreasing between 4% to 8%, and AUC ROC decreasing between 6& to 10% when compared to models trained upon non-feature suppressed data.

## 5.3 Application of Pre-processing Bias Mitigation

When applying the following Pre-processing Bias Mitigation algorithms, I used the following procedure for obtaining the performance and fairness metrics for each model and each pre-processing algorithm tested:

Similarly to the naive, and feature suppressed models, I obtained a 70/30 train-test split of the dataset. At this point, I applied each respective pre-processing algorithm to the training dataset, obtaining an adjusted dataset with which models can learn a more fair model. Following this, I ran 10-fold cross-validation upon the training dataset in order to obtain the performance metrics as seen in Tables 5.4 and 5.5. Finally, I then obtained a set of predictions for the held-out test set. The tables referenced above also contain the fairness metrics as calculated after comparing the predictions from each pre-processing algorithm to ground truth of the test dataset.

### 5.3.1 Reweighing

Table 5.4: Metrics as calculated on the test dataset predictions where Reweighing pre-processing algorithm applied.

| ***Results from application of Reweighing Pre-processing Mitigation Algorithm*** | | | | | | | |
|---|---|---|---|---|---|---|---|
| | **Performance Metrics** | | | **Fairness Metrics** | | | |
| **Model** | Accuracy | Precision | AUC ROC | DI | abs(1-DI) | SPD | GEI |
| RFC | 0.70 (+/- 0.08) | 0.69 (+/- 0.04) | 0.76 (+/- 0.10) | 1.097 | 0.097 | 0.009 | 0.825 |
| LR | 0.68 (+/- 0.11) | 0.69 (+/- 0.08) | 0.76 (+/- 0.14) | 0.994 | 0.006 | -0.003 | 0.333 |
| MLP | 0.69 (+/- 0.09) | 0.68 (+/- 0.07) | 0.77 (+/- 0.13) | 1.065 | 0.065 | 0.050 | 0.135 |
| GB | 0.64 (+/- 0.04) | 0.63 (+/- 0.03) | 0.70 (+/- 0.05) | 1.097 | 0.097 | 0.097 | 0.589 |
| AB | 0.64 (+/- 0.04) | 0.63 (+/- 0.03) | 0.69 (+/- 0.06) | 0.944 | 0.056 | -0.023 | 0.425 |

When comparing the results obtained for the Reweighing pre-processing bias mitigation algorithm shown in Table 5.4 to those obtained from the naive models in Table 5.2, there seems to be no appreciable difference in performance. As a result, when considering the trade-off between fairness and performance, any increase in fairness would be be much appreciated as performance wasn't impacted by the application of Reweighing.

Across all five models there was a decrease in $|1 - DI|$ value, meaning that all become more group fair after the application of Reweighing. In particular, there were large improvements for the three models that violated the 4/5ths Rule before any bias mitigation was applied. As a result, all five models after applying Reweighing now adhere to the 4/5ths Rule.

For the RFC, MLP, and GB classifiers, after applying Reweighing there was a slight bias in favour of males, as the DI and SPD scores were greater than 1.0 and 0.0, respectively. Similarly to when I removed sensitive features in the experiment above. However, the magnitude of this bias is much smaller than above, and in all cases smaller than the bias against males measured before any bias mitigation was applied.

In terms of SPD scores, all models aside from the MLP scored much better, with the RFC and LR classifiers having scores that corresponded in a less than 1% difference in outcome between males and females. This could be related to the Reweighing algorithm optimising for SPD score, as noted in Chapter 5 above. As for the MLP classifier, there was an improvement that corresponded to 1.5% better group fairness. However, other models saw a dramatic change in score, most notably the AB classifier from an SPD score of -0.340 to -0.023.

After the application of Reweighing, all models with the exception of MLP saw large increases in their GEI scores. In general, it can be inferred that while Reweighing has made all the models much more group fair (particularly in the cases of LR, GB, and AB models), there has been a large impact on individual fairness as seen in the impact on their GEI scores. The only exception being in the case of the MLP classifier, where the GEI score improved slightly, from 0.190 before applying Reweighing to 0.135 after.

Overall, the only classifier that improved in terms of both group and individual fairness after applying Reweighing was the Multi-Layer Percepton. This can be seen by its decreased Disparate Impact and Statistical Parity difference scores (in absolute terms), and its corresponding decrease in Group Entropy Index. While none of the other four models were able to improve upon their individual fairness, as shown by their increased GEI scores, they generally made substantial improvements in terms of group fairness. This was shown by their respective improvements in DI and SPD scores.

### 5.3.2 Disparate Impact Remover

Similarly to when applying the Reweighing algorithm, there seems to be no appreciable difference in performance shown in Table 5.5 as compared to Table 5.2 when applying the Disparate Impact Remover algorithm. Again, when considering the fairness and performance trade-off, any increase in fairness is appreciated as performance

Table 5.5: Metrics as calculated on the test dataset predictions where Disparate Impact Remover pre-processing algorithm applied

| ***Results from application of Disparate Impact Remover Pre-processing Mitigation Algorithm*** | | | | | | | |
|---|---|---|---|---|---|---|---|
| | **Performance Metrics** | | | **Fairness Metrics** | | | |
| **Model** | Accuracy | Precision | AUC ROC | DI | abs(1-DI) | SPD | GEI |
| RFC | 0.70 (+/- 0.07) | 0.69 (+/- 0.04) | 0.76 (+/- 0.10) | 0.980 | 0.020 | -0.002 | 0.797 |
| LR | 0.68 (+/- 0.11) | 0.69 (+/- 0.08) | 0.75 (+/- 0.14) | 0.963 | 0.037 | -0.009 | 0.651 |
| MLP | 0.69 (+/- 0.09) | 0.68 (+/- 0.06) | 0.76 (+/- 0.13) | 0.991 | 0.009 | -0.005 | 0.277 |
| GB | 0.64 (+/- 0.04) | 0.63 (+/- 0.03) | 0.70 (+/- 0.05) | 1.075 | 0.075 | 0.012 | 0.748 |
| AB | 0.64 (+/- 0.04) | 0.63 (+/- 0.03) | 0.69 (+/- 0.06) | 0.944 | 0.056 | -0.023 | 0.425 |

wasn't impacted when applying the Disparate Impact Remover algorithm.

There is one case here where there is a very slight bias in favour of males after the application of the Disparate Impact Remover algorithm. The GB classifier scores a positive 0.012 for SPD, and 1.075 for DI. However, when comparing these values to those of the naive GB classifier shown in Table 5.2, this nevertheless represents a great improvement.

For all five models, there was a significant decrease in $|1-DI|$ value, meaning all models became more group fair after applying the Disparate Impact Remover to the data. Similarly to above, all five models adhere to the 4/5ths rule. With the exception of the LR classifier, the Disparate Impact scores were equal to/improved again over the scores observed after applying the Reweighing algorithm. As noted in Chapter 5, the Disparate Impact Remover seeks to minimise the quantity of Disparate Impact in the dataset. This may explain why generally the DI scores are even lower than those observed in the results from applying Reweighing.

As for the SPD scores, all five models made a marked improvement after the application of the Disparate Impact remover. With the RFC, LR, and MLP classifiers all having scores corresponding to a less than 1% favourability towards females, and the GB classifier having a 1.2% favourability towards males. Although the Disparate Impact Remover algorithm is not optimised for reducing SPD; as the DI and SPD terms are so closely related, it made great improvements on both simultaneously.

Unfortunately, as was observed in the application Reweighing, the GEI scores for the models deteriorated after applying the Disparate Impact Remover algorithm. Unlike above, the MLP classifier did not buck the trend set by the other four models, as in this case its GEI score also increased.

Overall, when applying the Disparate Impact Remover algorithm, none of the five classifiers were able to improve their fairness with respect to both individuals and groups. This can be observed in the deterioration of all their Generalised Entropy Index scores. However, similarly to when Reweighing was applied, all five models substantially improved in terms of group fairness alone. Shown in the considerable improvements made upon their Disparate Impact and Statistical Parity Difference scores.

## 5.4 Application of In-processing Bias Mitigation

Here, I also obtained a 70/30 train-test split of the data, however no further pre-processing of data occurred. For in-processing bias mitigation algorithms attempt to produce a "fairness-aware" model from the original data. In this instance, I didn't run cross-validation, hence the performance metrics reported are obtained directly from the predictions obtained from the test dataset. However, the fairness metrics were calculated in the same way as in above sections, via the output from the predictions the model made of the test dataset.

### 5.4.1 Adversarial Debiasing

Table 5.6: Metrics as calculated on the test dataset predictions where Adversarial Debiasing in-processing bias mitigation algorithm applied.

| ***Results from Adversarial Debiasing In-processing Bias Mitigation Algorithm*** | | | | | | |
|---|---|---|---|---|---|---|
| **Performance Metrics** | | | **Fairness Metrics** | | | |
| Accuracy | Precision | AUC ROC | DI | abs(1-DI) | SPD | GEI |
| 0.639 | 0.723 | 0.575 | 0.962 | 0.038 | -0.056 | 0.170 |

Here performance scores - as seen in Table 5.6 are roughly equivalent to the other five models tested, with the exception of its AUC ROC score which was considerably lower. However, these were the scores as reported on the held-out test set, as I didn't get the opportunity to run cross-validation upon the Adversarial Debiasing model. As a result, this set of performance metrics is not as robust as others.

In terms of fairness metrics (shown in Table 5.6), the model learned by the Adversarial Debiasing algorithm performs equivalently to the Multi-Layer Perceptron's baseline, with slight improvements when compared to it across all three fairness metrics. Both models are produced via artificial neural networks, hence they learn using similar means, which may explain the similarities. Hence, it also well outperformed the baselines of each of the other four models tested, with a $|1 - DI|$ value of 0.038 and SPD score corresponding to a 5.6% favourability towards females.

When comparing the fairness of Adversarial Debiasing against those of the MLP classifier with various bias mitigation algorithms applied: it produces a better Group Entropy Index in all cases except compared to when Reweighing is applied; better Disparate Impact scores in all cases except compared to when the Disparate Impact Remover is applied; and better Statistical Parity Scores than all except when the Disparate Impact Remover, and Reject Option Post-processing algorithms are applied. Given Adversarial Debiasing's close to ideal scores across all three fairness metrics, it provides a great balance between group and individual fairness.

## 5.5 Application of Post-processing Bias Mitigation

When applying post-processing bias mitigation algorithms, there is no further pre-processing required after obtaining the 70/40 train-test split of the dataset - similarly to when using in-processing bias mitigation tactics. At this point I used the same process as when finding the performance metrics for the naive models, as for post-processing methods the bias is mitigated after the models are trained. Hence, I performed 10-fold cross-validation on the training set to obtain the performance metrics reported in Tables 5.7 and 5.8. After obtaining a set of predictions for each model upon the held-out test dataset, I applied each of the post-processing algorithms and calculated the fairness metric scores reported in the tables referenced above.

### 5.5.1 Calibrated Equalised Odds Post-processing

Table 5.7: Metrics as calculated on the test dataset predictions where Calibrated Equalised Odds post-processing bias mitigation algorithm applied.

| ***Results from Calibrated Equal Odds Post-processing Bias Mitigation Algorithm*** | | | | | | | |
|---|---|---|---|---|---|---|---|
| | **Performance Metrics** | | | **Fairness Metrics** | | | |
| **Model** | Accuracy | Precision | AUC ROC | DI | abs(1-DI) | SPD | GEI |
| RFC | 0.70 (+/- 0.08) | 0.69 (+/- 0.05) | 0.77 (+/- 0.10) | 0.875 | 0.125 | -0.091 | 0.204 |
| LR | 0.68 (+/- 0.12) | 0.69 (+/- 0.08) | 0.76 (+/- 0.14) | 0.749 | 0.251 | -0.234 | 0.156 |
| MLP | 0.69 (+/- 0.10) | 0.68 (+/- 0.06) | 0.77 (+/- 0.13) | 0.670 | 0.330 | -0.330 | 0.173 |
| GB | 0.65 (+/- 0.05) | 0.63 (+/- 0.03) | 0.71 (+/- 0.06) | 0.738 | 0.262 | -0.185 | 0.251 |
| AB | 0.64 (+/- 0.05) | 0.63 (+/- 0.03) | 0.70 (+/- 0.07) | 0.504 | 0.496 | -0.496 | 0.244 |

As mentioned above, applying post-processing bias mitigation algorithms entails adjusting the output from the naive classifiers. Hence, the performance metrics reported in Table 5.7 are equivalent to those shown in Table 5.2.

Looking at the fairness metrics as reported in Table 5.7 after applying Calibrated Equalised Odds post-processing, it's clear to see that there was no improvement across any scores upon any of the metrics. Overall, in this application, it seems that CEOP was ineffective. When proposed by Pleiss et al. in [22], the paper expressed some concerns regarding its effectiveness, and unfortunately these concerns seem to have been borne out during my experiment.

### 5.5.2 Reject Option Post-processing

Again, the performance metrics reported in Table 5.8 are equivalent to those shown in Table 5.2. As post-processing bias mitigation is applied to the output from the naive classifiers.

With the exception of the AdaBoost classifier, which I will discuss separately, there was substantial improvements in terms of group fairness for each model when Reject

Table 5.8: Metrics as calculated on the test dataset predictions where Reject Option Classification post-processing bias mitigation algorithm applied.

| ***Results from Reject Option Classification Post-processing Bias Mitigation Algorithm*** | | | | | | | |
|---|---|---|---|---|---|---|---|
| | **Performance Metrics** | | | **Fairness Metrics** | | | |
| **Model** | Accuracy | Precision | AUC ROC | DI | abs(1-DI) | SPD | GEI |
| RFC | 0.70 (+/- 0.08) | 0.69 (+/- 0.05) | 0.77 (+/- 0.10) | 1.049 | 0.049 | 0.024 | 0.282 |
| LR | 0.68 (+/- 0.12) | 0.69 (+/- 0.08) | 0.76 (+/- 0.14) | 0.925 | 0.075 | -0.036 | 0.323 |
| MLP | 0.69 (+/- 0.10) | 0.68 (+/- 0.06) | 0.77 (+/- 0.13) | 0.935 | 0.065 | -0.040 | 0.248 |
| GB | 0.65 (+/- 0.05) | 0.63 (+/- 0.03) | 0.71 (+/- 0.06) | 0.937 | 0.063 | -0.033 | 0.280 |
| AB | 0.64 (+/- 0.05) | 0.63 (+/- 0.03) | 0.70 (+/- 0.07) | inf | inf | 0.035 | 0.944 |

Option post-processing was applied. This is shown in each model's respective $|1 - DI|$ values, and SPD scores, all tending towards 0. The most marked differences being in the group fairness scores of the LR and GB models, where their naive classifiers did not adhere to the 4/5ths Rule. Respectively, the DI and SPD scores improved from 0.752 and -0.231, to 0.925 and -0.036 for the LR model. With an improvement from 0.738 and -0.185, to 0.937 and -0.033 for the GB model. Hence, each of the four models other than the AB classifier all adhere at the 4/5ths Rule after the application of Reject Option post-processing.

As seen in Table 5.8, the Disparate Impact score for the AdaBoost classifier has been marked as infinite. This was due to the Reject Option post-processing algorithm producing a selection rate for instances classed as female equal to 0. Meaning that when adjusting the predictions made by the AB classifier it did not predict a favourable outcome for any of the female instances in the test set. As a result, when attempting to calculate Disparate Impact there was a division by zero error, resulting in an infinite output.

Upon discovering this error, I attempted to remedy it by testing each variation of optimisation metric available for applying Reject Option post-processing. For the other four models, the optimal score was achieved when the optimisation metric applied was "Equal Opportunity Difference", which is equal to the difference in True Positive Rate between the privileged and unprivileged group. At this point I tested the other two optimisation metrics available to Reject Option post-processing, "Statistical Parity Difference", and "Average Odds Difference". Where Statistical Parity Difference is the same quantity as discussed in this report, and Average Odds Difference being equivalent to the average difference between False Positive Rate and True Positive Rate for the privileged and unprivileged groups. However, neither of these optimisation metrics were able to resolve the issue, both also producing the same error as seen in Table 5.8.

Given that each model has a higher Group Entropy Index score than their respective naive classifiers, it can be concluded that the Reject Option post-processing algorithm produces less individual fairness. As despite improvement across the board in scores for group fairness metrics, Disparate Impact and Statistical Parity Difference, each model saw a deterioration in GEI score.

Despite improvement in group fairness across four of the five models tested, there was

a fundamental error when applied to the fifth model and a deterioration across the board in Group Entropy Index scores. Consolidating all of the above, I'd deem Reject Option post-processing as having been ineffective in this application.

Ultimately, the main factor to be considered when choosing where in the pipeline to apply bias mitigation comes down to where it is feasible to do so. As some developers may not have access to the training data, meaning they cannot manipulate it using a pre-processing tactic. Some may use a "black-box" classification algorithm, having no ability to change the model itself, hence in-processing becomes unsuitable. Furthermore, some may not be able to do either of the above, limiting them to only post-processing strategies.

# Chapter 6

# Conclusion

Algorithmic Bias is a huge problem facing industry, one that requires a solution in order to preserve public trust in AI. Despite there having been many exposures of how algorithms can serve to propagate human bias, there seems to be a lack of widespread exposure to the subject of algorithmic fairness. Fairness-aware AI is a burgeoning field of research, and requires the attention of the community of developers if we are to ensure that automated decsion making processes become fair and representative.

But where does algorithmic bias stem from? The mantra *garbage in, garbage out* neatly summarises how the use of poor training data results in bias. This can come from the form of using data that encapsulates an historic human bias, from not using data that is representative of the domain it's learning for, or from allowing the omission of important attributes.

I have shown in this report that there exists a wide range means and methods available for us to utilise in the interests of quantifying and mitigating algorithmic bais within our automated decision making processes. Additionally, I have introduced the reader to a great example of an open source project, AI Fairness 360, which seeks to provide a platform for developers to learn about and contribute to solving the problem of algorithmic bias. Furthermore, I have provided a case study wherein I apply the above, to give the reader some perspective of what results to expect and how to interpret them.

# Chapter 7

# Future Work

If I were to continue to work on this project, I'd extend it to contribute to the AI Fairness 360 open source project. I noted that several pre-processing bias mitigation methods introduced by Kamiran & Calders in [14] have not been implemented. Additionally, there is research into using a variant of k-Nearest Neighbours Classification as a means of testing for and removing discrimination as introduced by Luong et al. in [18] that would be interesting to research and provide an implementation of. After implementation, I would spend some time benchmarking my contributions against existing implementations in the project to offer some analysis of how performant they are.

Additionally, this report is by no means exhaustive, as the field of "fairness-aware" AI and algorithmic fairness is extremely active and in a somewhat developmental stage. Hence, I could add to my work in this field through continued research and adding to the case study outlined in this report.

# Bibliography


[1] *Equality Act (2010)*, April 2010.

[2] J. Angwin, J. Larson, L. Kirchner, and S. Mattu. *Machine Bias: Theres software used across the country to predict future criminals. And its biased against blacks.*, May 2016.

[3] The American Bar Association. *Disparate Impact: Unintentional Discrimination*, January 2018.

[4] R.K.E. Bellamy, K. Dey, M. Hind, S.C. Hoffman, S. Houde, K. Kannan, P. Lohia, J. Martino, S Mehta, A. Mojsilovic, S. Nagar, K.N. Ramamurthy, J. Richards, D. Saha, P. Sattigeri, M. Singh, K.R. Varshney, and Y. Zhang. *AI Fairness 360: An Extensible Toolkit for Detecting, Understanding, and Mitigating Unwanted Algorithmic Bias*, October 2018.

[5] T. Calders and S. Verwer. *Three naive Bayes approaches for discrimination-free classification*. *Data Mining and Knowledge Discovery*, 21:277–292, September 2010.

[6] T. Calders and I. Zliobaite. *Why Unbiased Computational Processes Can Lead to Discriminative Decision Procedures*, pages 43–57. 01 2013.

[7] N.V. Chawla, K.W. Bowyer, L.O. Hall, and W.P. Kegelmeyer. *SMOTE: Synthetic Minority Over-sampling Technique*. 2011.

[8] Science & Technology Committee. *Algorithms in Decision Making*, May 2018. HC 351, para. 13.

[9] D. Danks and A.J. London. *Algorithmic Bias in Autonomous Systems*. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI-17*, pages 4691–4697, 2017.

[10] J. Dastin. *Amazon scraps secret AI recruiting tool that showed bias against women*, October 2018.

[11] C. Dwork, M. Hardt, T. Pitassi, O. Reingold, and R. Zemel. *Fairness Through Awareness*, 2011.

[12] M. Feldman, S. Friedler, J. Moeller, C. Scheidegger, and S. Venkatasubramanian. *Certifying and removing disparate impact*, 2014.

[13] M. Hardt, E. Price, and N. Srebro. *Equality of Opportunity in Supervised Learning*, 2016.

[14] F. Kamiran and T. Calders. *Data preprocessing techniques for classification without discrimination*. *Knowledge and Information Systems*, 33(1):1–33, Oct 2012.

[15] F. Kamiran, A. Karim, and X. Zhang. *Decision Theory for Discrimination-Aware Classification*. pages 924–929, 12 2012.

[16] W. Koehrsen. *Beyond Accuracy: Precision and Recall*, May 2018.

[17] J. Langston. *Whos a CEO? Google image results can shift gender biases*, April 2015.

[18] B.T. Luong, S. Ruggieri, and F. Turini. *k-NN As an Implementation of Situation Testing for Discrimination Discovery and Prevention*. In *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '11, pages 502–510. ACM, 2011.

[19] K. Martin. *Ethical Implications and Accountability of Algorithms*. *Journal of Business Ethics*, Jun 2018.

[20] S. Narkhede. *Understanding AUC - ROC Curve*, June 2018.

[21] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, M. Brucher D. Cournapeau, M. Perrot, and E. Duchesnay. *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[22] G. Pleiss, M. Raghavan, F. Wu, J. Kleinberg, and K.Q. Weinberger. *On Fairness and Calibration*, 2017.

[23] M. Reynolds. *Biased policing is made worse by errors in pre-crime algorithms*, October 2017.

[24] O. Schwartz. *Untold History of AI: Algorithmic Bias Was Born in the 1980s*, April 2019.

[25] T. Speicher, H. Heidari, N. Grgic-Hlaca, K.P. Gummadi, A. Singla, A. Weller, and M. Bilal Zafar. *A Unified Approach to Quantifying Algorithmic Unfairness: Measuring Individual & Group Unfairness via Inequality Indices*. 2018.

[26] B. Spice. *Questioning the Fairness of Targeting Ads Online*, July 2015.

[27] J. Vincent. *Google fixed its racist algorithm by removing gorillas from its image-labeling tech*, January 2018.

[28] J. Vincent. *Gender and racial bias found in Amazons facial recognition technology (again)*, January 2019.

[29] P. Wu. *GDPR and its impacts on machine learning applications*, November 2017.

[30] B.H. Zhang, B. Lemoine, and M. Mitchell. *Mitigating Unwanted Biases with Adversarial Learning*, 2018.